```
# Cell 12 - Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import FastMarkerCluster
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
%matplotlib inline
# Cell 13 - Load data
df = pd.read_csv("/content/AB_NYC_2019.csv (2).zip")
# Cell 14 - First rows
df.head()
# Cell 15 - Last rows
df.tail()
# Cell 16 - Describe
df.describe()
```

```
# Cell 17 - Duplicates check
df.duplicated().sum()
# Cell 18 - Drop duplicates
df.drop_duplicates(inplace=True)
# Cell 19 - Missing values check
df.isnull().sum()
# Cell 20 - Outliers - Price boxplot
plt.figure(figsize=(10,5))
sns.boxplot(df['price'])
plt.show()
# Cell 21 - Fill missing + drop last_review
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
df = df.drop(columns=['last_review'])
# Cell 22 - Drop unwanted columns
df = df.drop(columns=['id', 'name', 'host_id', 'host_name'])
# Cell 23 - Remove outliers - 95% quantile
price_limit = df['price'].quantile(0.95)
nights_limit = df['minimum_nights'].quantile(0.95)
df = df[(df['price'] <= price_limit) & (df['minimum_nights'] <= nights_limit)]
```

```
# Cell 24 - Check shape after cleaning
print(df.shape)
print(df.isnull().sum())
# Cell 25 - Drop neighbourhood + check
df = df.drop(columns=['neighbourhood'])
df.head()
# Cell 26 - One Hot Encoding
df = pd.get_dummies(df, columns=['neighbourhood_group', 'room_type'], drop_first=True,
df.head()
# Cell 27 - Check columns / shape / nulls
print(df.columns)
print(df.shape)
print(df.isnull().sum())
# Cell 28 - X and y split
X = df.drop('price', axis=1)
y = df['price']
print("Features shape:", X.shape)
print("Target shape:", y.shape)
# Cell 29 - Train Test Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
# Cell 30 - Linear Regression Model
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print("Linear Regression Model Trained Successfully!")
# Cell 31 - Prediction
y_pred = model.predict(X_test)
print("Predicted Prices:")
print(y_pred[:10])
# Cell 33 - Evaluation Metrics
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print("R-Squared Score:", r2)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
# Cell 34 - Actual vs Predicted Graph
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linesty
```